

PROJECT REPORT

Premier League match-outcome model

Three-class (home / draw / away) match prediction from 25 seasons of results and pre-season squad values: gradient-boosted trees that start from the bookmaker's price, blended with a Dixon–Coles goals model, and judged by log loss against the market itself.

Every number below is read from a committed artefact of the repository (the model config, the exported predictions and metrics, the ablation file, or statistics derived from the raw season files by a script). Section 15 lists which.

1 Summary

The model predicts the full-time result of a Premier League match, home win (H), draw (D) or away win (A), using only information available before kickoff. On the **test** split (2023-24 and 2024-25, 699 matches) it reaches 56.1% accuracy against a 43.9% always-home baseline, with a log loss of 0.946 against the de-vigged bookmaker line's 0.949. On the **holdout** split (2025-26, 350 matches, evaluated once) accuracy is 48.3% against 42.0%, but log loss is 1.019 against the market's 1.016: marginally behind the price.

The honest reading is that the model matches a settled market out of sample rather than beating it. Its dominant limitation is that under argmax it **never predicts a draw**: draw recall is 0.0% on test (163 drawn matches) and on holdout (98), so all of its accuracy comes from separating home wins from away wins. Sections 3 and 11 explain why that is expected for a model judged on probabilities, and what it costs.

0.946

test log loss · market 0.949

1.019

holdout log loss · market 1.016

56.1%

test accuracy · always-home 43.9%

0.0%

draw recall, test and holdout

2 The problem

Match prediction is a three-class problem in which one class has almost no distinct signal.

Across the 9,074 matches in the filtered dataset the outcome mix is 46.0% home, 24.7% draw, 29.3% away, so predicting "home" every time scores about 46% for free. In the evaluation windows the home share is lower, 43.9% on test and 42.0% on holdout, but the baseline is high enough that accuracy alone is a weak measure of skill.

Draws are structurally harder than the other two classes. A draw is not a strategy a team executes; it is what happens when two low scoring rates collide and produce 0–0 or 1–1. It has no feature signature of its own: it occupies the region between "home is stronger" and "away is stronger", exactly where the outcome is least determined. Section 6 shows this directly: the correlation of every pre-kickoff feature with a draw sits between -0.05 and $+0.05$. A classifier minimising log loss will rationally hold the draw probability near its base rate and almost never make it the largest of three.

The real benchmark is the bookmaker. A de-vigged consensus 1X2 price aggregates every model, injury report and stake in the market; it scores 0.949 log loss on test and 1.016 on holdout. Beating always-home is easy; beating the closing line is the actual test, and this project does not clearly pass it. What it does show is how close a small, leak-free model built from public statistics can get, and where the remaining gap sits.

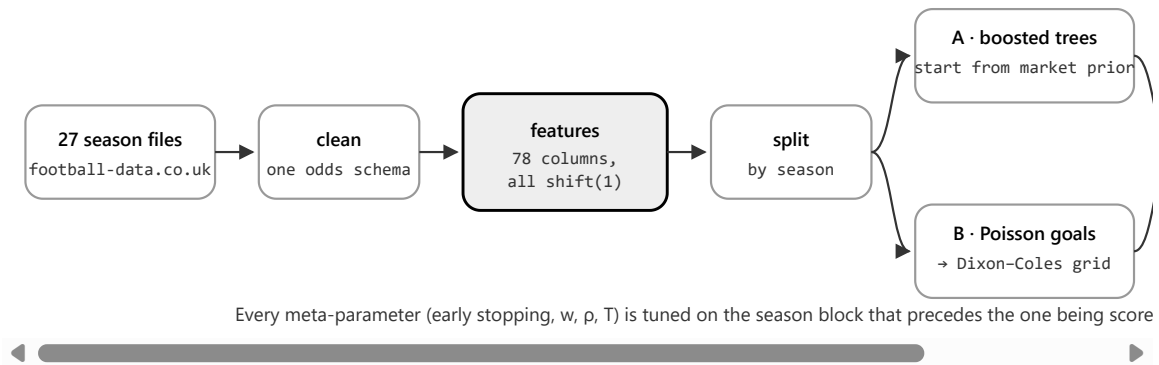


Figure 1 The pipeline. Cleaning collapses 25 years of changing bookmaker columns into one schema; feature engineering turns each match into 78 pre-kickoff differences; two models see the same features and are combined at the end.

3 Why log loss, not accuracy

Accuracy asks one question: was the most likely class the one that happened? For a three-way outcome with a 46% base rate that question is nearly uninformative, and it is blind to everything the model actually produces. A model that says 51/24/25 and a model that says 90/5/5 both "predict home"; if the match is drawn, accuracy punishes them equally, though the second was far more wrong.

Log loss grades the probability the model assigned to what actually happened:

$$LL = - (1/N) \sum_i \log p_i(y_i)$$

where $p_i(y_i)$ is the probability the model gave to the outcome that occurred in match i . Confident and right costs almost nothing; confident and wrong is punished hard. A flat one-third guess scores $\log 3 = 1.099$.

Three consequences drive the design. First, log loss is a *proper scoring rule*: the way to minimise it is to report your true belief, so it rewards honest probabilities and makes calibration (section 11) a first-class concern rather than an afterthought. Second, it puts the model and the bookmaker on the same scale: the market publishes probabilities, not labels, and the only fair comparison is probability against probability. Third, it changes what "handling the draw" means. A calibrated 28% draw probability on a tight match is *credit* under log loss even though the argmax still says home; under accuracy that same probability is invisible. The

project therefore treats a well-calibrated P(draw) as the deliverable and reports the argmax accuracy as a secondary, coarser view.

The ranked probability score is reported alongside as a check that respects the ordering H, D, A; it tells the same story (section 13).

4 Data

Source. football-data.co.uk [1] season files, 2000-01 through 2026-27. Seasons before 2000-01 carry no shot, corner or card columns, so a statistics-based model cannot use them.

Volume. 9,900 matches with a result parse from the files. Rows where either team has fewer than three prior matches in the current season are dropped, since cold-start rows have no meaningful form; 826 go, including every opening fixture of every season, and 9,074 remain.

Kept per match: date, both teams, full-time goals and result, and the twelve match-statistic columns (shots, shots on target, corners, fouls, yellows, reds, per side). Real expected goals exist only in the 2026-27 file and are used where present.

Dropped, and why. The half-time result occurs after kickoff and cannot be an input to its own match. The referee is noise for outcome prediction. The forty-odd raw bookmaker columns change provider and name every few seasons; rather than feed the model a sparse matrix of era-specific columns, they are collapsed into one stable schema, taking the first fully populated provider group per row: a consensus 1X2 price, an over/under 2.5 price, and a separate closing 1X2 price with Pinnacle preferred.

Supplementary source. Per-season Fantasy Premier League rosters and gameweek-1 price snapshots from FPL Core Insights [2]. This data begins at 2024-25, so squad features are missing for every earlier season; XGBoost routes missing values natively, so the sparse coverage costs nothing on older rows.

5 Leakage, and the rule that prevents it

The governing constraint: **a feature for match k may use only matches $1 \dots k-1$.** Every design decision downstream traces to this rule. Every rolling, expanding and exponentially weighted statistic is shifted by one match before it is aggregated. Elo is recorded pre-update: the rating on a match row is the one the team carried *into* that match, never the one it left with. Odds are the pre-kickoff price; the closing line enters only as separate features describing where the market moved, itself known before kickoff. Squad prices are frozen at each season's gameweek-1

snapshot. Final-score goals enter only as the target of the Poisson goal model, never as an input.

What leakage would look like here

There are two ways this dataset leaks, and both are quiet. The first is a random train/test split. Rolling features summarise the matches before each row, so rows are not independent: row k carries information about rows $1 \dots k-1$. A shuffle puts later matches in training and earlier matches in test; the model learns from the future to predict the past and its test score inflates. The cure is a chronological cut by season (section 12).

The second is the order of two pandas operations. `rolling(3).mean()` at match k includes match k . `shift(1).rolling(3).mean()` averages only the three matches before it. Man United's first eight league matches of 2015-16, from the season file, with a window of three:

THE SAME FEATURE, COMPUTED THE WRONG WAY AND THE RIGHT WAY

Date	Opponent	Pts	roll-3 (leaks)	shift(1) + roll-3 (safe)
2015-08-08	Tottenham (H)	3	3.00	—
2015-08-14	Aston Villa (A)	3	3.00	3.00
2015-08-22	Newcastle (H)	1	2.33	3.00
2015-08-30	Swansea (A)	0	1.33	2.33
2015-09-12	Liverpool (H)	3	1.33	1.33
2015-09-20	Southampton (A)	3	2.00	1.33
2015-09-26	Sunderland (H)	3	3.00	2.00
2015-10-04	Arsenal (A)	0	2.00	3.00

Row three is the leak in miniature. The Newcastle match was a 0–0 draw worth one point. The leaky column reads 2.33 because it has already averaged in that point: the feature knows the result it is supposed to predict. The safe column reads 3.00, the mean of the two wins before it. The first row is empty by construction: no prior match exists. A model trained on the left column would discover that "teams who did well in a match did well in it", look brilliant in a notebook, and collapse on real fixtures.

The constraint is verified, not assumed. The feature notebook recomputes one rolling feature independently for 150 sampled matches and asserts a maximum difference of zero, and checks that both teams in the dataset's first match carry an

Elo of exactly 1500. The two quantities fitted on the data rather than computed from it, the expected-goals proxy (section 6) and the odds-bias exponent (section 7), are fitted on seasons up to 2018-19 only, so nothing downstream of the evaluation splits touches them.

6 Features

78 features, almost all differences of the form *home value* – *away value*, in six groups.

Rolling form and expected goals. Last-five and exponentially weighted (half-life 4) means of points, goals for and against, goal difference, shots, shots on target and corners; season-to-date expanding means that reset each August; a venue-restricted points average splitting home form from away form; shot conversion and accuracy. An expected-goals proxy is fitted by least squares of actual goals on off-target and on-target shots:

$$\hat{x}G = a + b \cdot (\text{shots} - \text{SoT}) + c \cdot \text{SoT}$$

solved from the 3×3 normal equations on seasons up to 2018-19; on-target shots carry almost all the weight, off-target shots almost none, the shape a real xG model has. Real xG replaces it where the source provides it.

Rolling xG, xG-against and xG-difference follow, plus a finishing term (goals minus xG over the last five) that flags hot or cold finishing and regresses to zero.

Elo. One rating per team, updated after every match, absorbing strength of schedule:

$$E_h = 1 / (1 + 10^{-(R_h + \text{HFA} - R_a)/400}), \quad R \leftarrow R + K \cdot \sqrt{\max(|\Delta \text{goals}|, 1)} \cdot (S - E)$$

K = 20; S is 1, ½ or 0 for a win, draw or loss. Three departures from textbook Elo: the update is margin-aware, so a three-goal win moves ratings about 1.7 times a one-goal win; the home-field advantage is not a fixed 60 points but the value implied by the previous three seasons' league-wide home points rate, which fell from 78.7 for 2011-12 to 24.7 for 2022-23 once the empty-stadium 2020-21 season entered the window; and ratings regress 25% toward 1500 each season boundary, with promoted teams entering at 1500.

Venue, rest and head-to-head. Home-versus-away form split; days since the previous match; matches played in the last 14 days; mean points in the last five meetings of this pair, keyed on an unordered pair so the history is symmetric.

League position and pressure. The table as it stood before each match (points, goal difference, rank), the points gap to 18th and to 4th, and the games-played gap.

Market signal (19 columns). De-vigged 1X2 probabilities two ways (proportional, and a power transform correcting favourite–longshot bias, fitted on 2000–2018); the overround; supremacy; entropy; implied total goals recovered by inverting the Poisson tail on the de-vigged over/under 2.5 line and split into implied home and away expected goals; closing-line probabilities; and closing-minus-opening movement.

Squad strength (9 columns). Total squad value, first-XI value, bench value, prior-season points per game across the first XI, best-keeper and per-line prices, and an attack-versus-defence edge term. 2024-25 onward only.

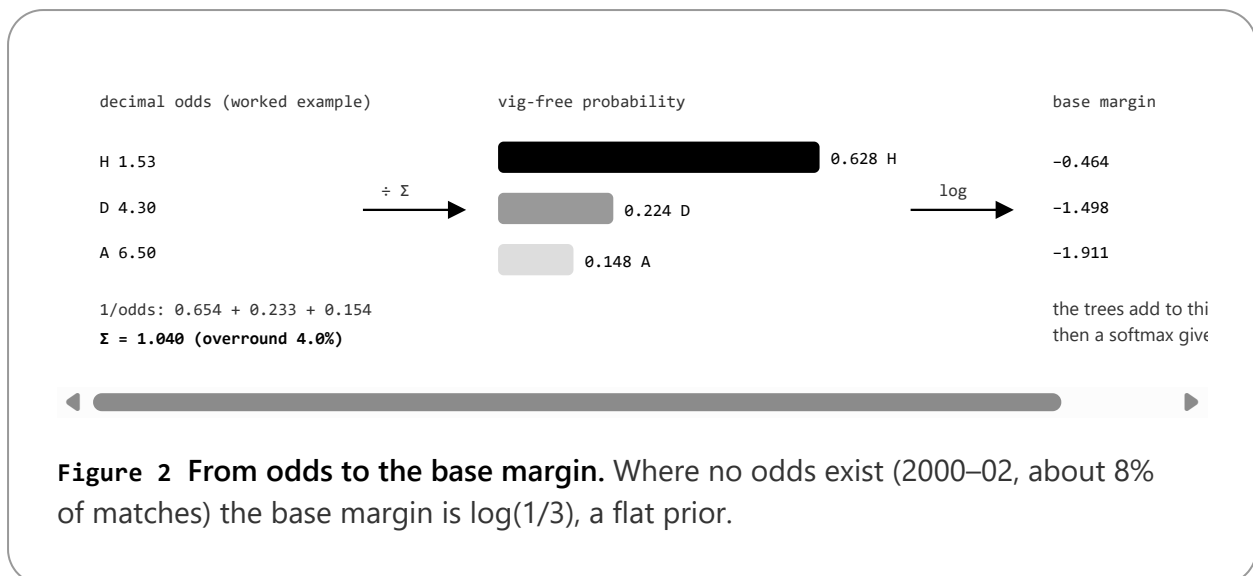
Before any model is trained, the correlation of each feature with each outcome already tells the story of section 2: Elo difference correlates at 0.35 with home wins and -0.35 with away wins, season-to-date goal difference and points close behind, and no feature correlates with a draw beyond ± 0.05 . The pipeline page draws the full table.

7 The market as a prior, not a competitor

The obvious framing is a contest: build a model, compare it with the bookmaker, hope to win. This project rejects that framing on the evidence. A closing line aggregates every public model, every team-sheet leak and every stake placed; over the seven rolling-origin seasons in section 13 its log loss ranged from 0.897 to 1.000, and no feature set built from public statistics matches it from a standing start. The ablation in section 13 makes the point numerically: adding the market columns to the form, Elo and xG features improves test log loss from 0.9491 to 0.9455, the largest single step on the ladder.

So the model uses the price as a *prior*. Its job is not to replace the market's estimate but to find the places where 25 years of results and 78 public numbers disagree with it, and move a few points in that direction. This has two practical merits. Where the model has nothing to add, it inherits the market's estimate instead of a worse one of its own, and where the model is uncertain, it has somewhere sensible to fall back to.

Raw odds do not quite work as probabilities: the inverses of the three prices sum to a little over one, the overround. Dividing by the sum gives vig-free probabilities. Taking their logs gives the log-odds the trees will start from.



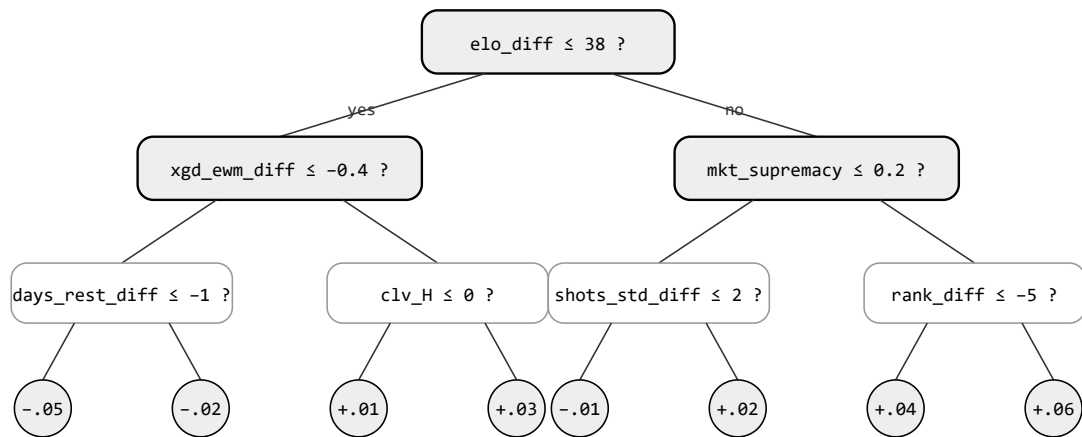
8 Model A: boosted trees on top of the prior

Gradient boosting fits a chain of small trees. Each tree is trained on what the chain so far still gets wrong, and its leaves are not verdicts but small additions to a running log-odds. XGBoost's `base_margin` is what makes the prior concrete: it is the value the running total starts from for each row, before any tree has spoken. Setting it to the log of the vig-free market probability means the trees only ever model the *residual*:

$$z_k(x) = \log p^{\text{mkt}}_k + \sum_{t=1..T} f_{t,k}(x), \quad p_k = \text{softmax}(z)_k$$

for classes $k = H, D, A$. With no trees at all the model is the market. Each tree f_t adds a correction where the features say the price is off.

The classifier is a `multi:softprob` XGBoost model with 3-deep trees, learning rate 0.03, row subsampling 0.85, column subsampling 0.70, minimum child weight 8, L2 2.0 and L1 0.5 regularisation, up to 4,000 rounds with early stopping on the validation block. There are **no class weights**. An earlier version used inverse-frequency weights to force draws up; it predicted draws it did not believe, and the log loss got worse. Boosting already minimises the multi-class log loss directly, so the probabilities are left honest and any draw-seeking is handled at the decision step (section 11).



Schematic: three questions deep, leaves are nudges to the running log-odds. Thresholds and values are illustrative.

Figure 3 One tree in the chain. A depth-3 tree asks at most three questions of a match and lands in one of eight leaves; the leaf's value is added to the log-odds, and the next tree starts from the updated total. A few hundred such trees, each scaled by the learning rate, make up Model A.

9 Model B: Dixon–Coles goals

A draw is not a class a team chooses; it is what happens when two low scoring rates meet. So Model B does not classify at all. Two XGBoost regressors with a Poisson objective predict the expected home goals λ and away goals μ , each starting from the market's implied expected goals as its base margin (the same prior idea as section 8, applied to a count). From those two numbers a full scoreline distribution follows:

$$P(i, j) = \text{Pois}(i; \lambda) \cdot \text{Pois}(j; \mu) \cdot \tau(i, j)$$

i home goals, j away goals. Summing the lower triangle ($i > j$) gives $P(H)$, the diagonal gives $P(D)$, the upper triangle gives $P(A)$. The draw falls out of the arithmetic.

Two independent Poissons are known to be wrong in one specific place: they under-predict 0–0 and 1–1 and over-predict 1–0 and 0–1, because goals in low-scoring games are not independent. Dixon and Coles (1997) fixed this with a correction on exactly those four cells:

$$\tau(0,0) = 1 - \lambda\mu\rho, \quad \tau(1,0) = 1 + \mu\rho, \quad \tau(0,1) = 1 + \lambda\rho, \quad \tau(1,1) = 1 - \rho, \quad \tau = 1 \text{ otherwise}$$

ρ is fitted on the tuning block by minimising outcome log loss; the committed value is -0.11 . A negative ρ raises the two draws and lowers the two one-goal wins, then the matrix is renormalised. Note that the sign convention here is the one the code uses: with $\rho < 0$, $1 - \lambda\mu\rho > 1$.

The same matrix supplies the exact-score grid the site shows for every match, and the over-2.5 and both-teams-score probabilities. Model B tends to be sharper than Model A on tight, low-scoring games, which is precisely where Model A is weakest; that complementarity is the reason for the blend.

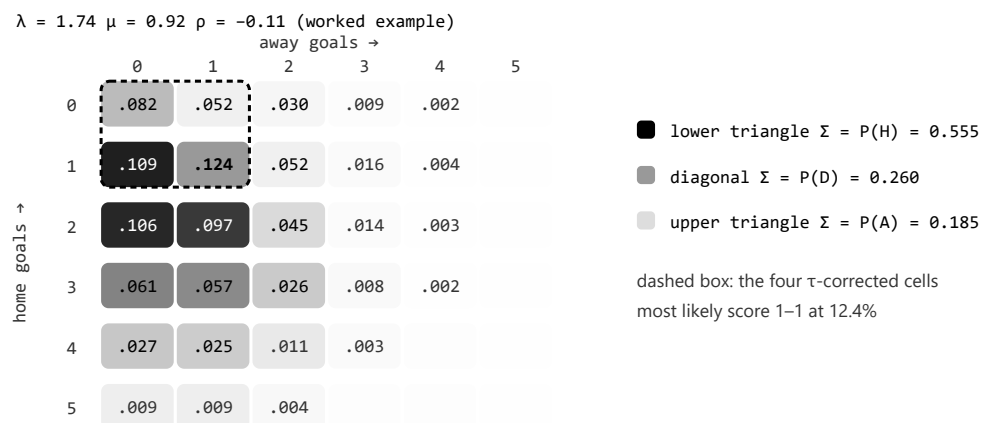


Figure 4 A Dixon–Coles scoreline matrix. Expected goals are chosen for illustration; ρ is the committed value and every cell is computed from it.

10 The blend

Model A and Model B see the same features but make different mistakes: the classifier is sharper on strong favourites, the goal model on tight, low-scoring games. When two forecasts have errors that are not fully correlated, their average has a lower error than either, which is the same reason a forest beats a single tree. The blend is a plain weighted mean of the two probability vectors:

$$p = w \cdot p_A + (1 - w) \cdot p_B$$

w is chosen by evaluating the log loss of the blend on the tuning block at 41 equally spaced values of w from 0 to 1 and keeping the minimum. Nothing is fitted; it is a one-dimensional search on held-out matches.

In the committed model $w = 0.30$: the classifier carries 30% of the vote and the goal model 70%, tuned on the 2023-25 block. That weight is not stable across tuning windows. The bundle tuned on 2021-23, which produced the test-season numbers, weighted the classifier at about 0.925, a figure reconstructed from the published test predictions by searching for the weight and temperature that reproduce them (mean absolute error 0.00003, rounding noise). Two seasons of validation, about 700 matches, pin the log loss surface only loosely along w , and the two models are close enough in quality that the optimum wanders. The practical reading is that the blend is worth having, and that its exact proportions should not be over-interpreted.

11 Calibration, and why the draw stays hard

A model can have a good log loss and still be systematically over- or under-confident: it says 65% and the event happens 58% of the time. Calibration is the post-processing step that maps stated probabilities onto observed frequencies. Two calibrators were fitted on the tuning block. A per-class isotonic map (pool-adjacent-violators, hand-rolled in numpy) is the flexible choice, but on 700 tuning matches it overfits the bumps of the validation sample. The one that ships is a single temperature:

$$p'_k \propto p_k^{1/T}$$

one parameter, chosen on the tuning block by grid search over T in $[0.5, 3.0]$. $T > 1$ softens every probability toward one third, $T < 1$ sharpens. The committed value is $T = 1.04$: the blend was already close to honest.

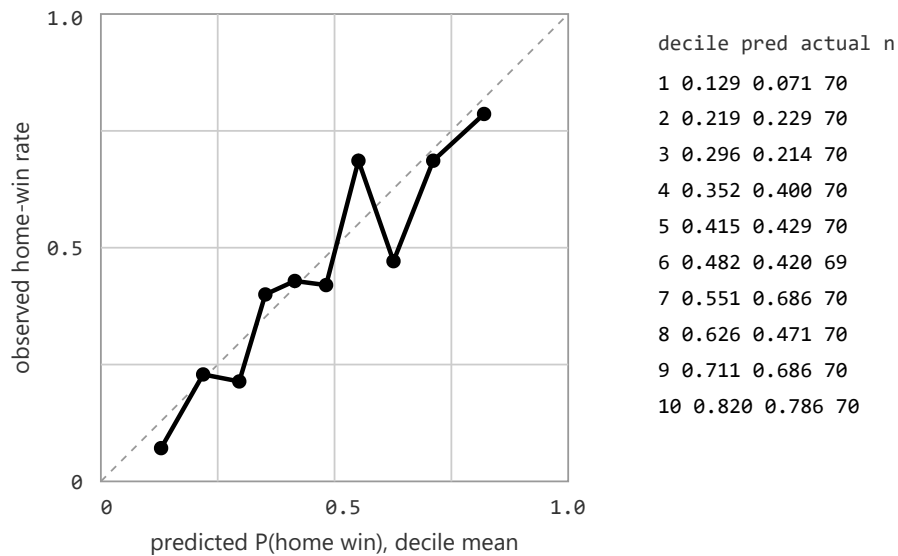


Figure 5 Reliability of P(home win) on the test seasons. Calibration is good at the extremes and unstable in the middle: the 0.551 bucket won 0.686 while the 0.626 bucket won 0.471, a non-monotonicity across about 70 matches per bucket, which is the size of sampling noise at that depth.

Why the draw stays hard

Three reasons stack up. **Base rate:** draws are about a quarter of matches, the smallest class, and a probability near 25% is almost never the largest of three. **No signature:** no pre-kickoff feature correlates with a draw, so the model's draw probability stays close to its base rate and the argmax never lands on it. **Geometry:** a draw sits between H and A in every feature that matters, so any small error pushes the prediction into the nearest decisive class.

None of this is a calibration failure. Temperature scaling maps stated probabilities to observed rates and does exactly that for the draw too: a 26% draw probability is right about 26% of the time. What calibration cannot do is invent a signal that is not in the features. For a use case that needs draws *called*, the project reports a decision rule rather than a modelling change: multiply $P(\text{draw})$ by 1.75 before the argmax, a factor chosen on the tuning block to reach 40% draw recall. It trades accuracy for draw recall and is never fed back into training.

12 Evaluation method

Chronological split. The data is cut by season, never shuffled. Counts are after the three-prior-games filter, derived from the season files by the same rule the notebook applies.

SPLITS

Slice	Seasons	Matches	Purpose
train	2000-01 ... 2020-21	7,325	fit the trees
valid	2021-22, 2022-23	700	early stopping, blend weight, p , temperature
test	2023-24, 2024-25	699	the slice looked at while iterating
holdout	2025-26	350	evaluated exactly once

The holdout is nominally 2025-26 and 2026-27, but no 2026-27 row yet clears the three-prior-games filter (20 matches played, two per club), so in practice it is 2025-26 only.

Two bundles. Each is tuned on data that precedes what it scores. One is fitted on train and tuned on valid to score test; the other, the committed model, is fitted on train plus valid and tuned on test to score the holdout. Early-stopping round, blend weight, p and temperature are all chosen on the tuning slice, never the scoring slice.

Rolling-origin check. One split can be lucky. For each of seven recent seasons the full blend is refitted on everything preceding that season's two-season tuning block and scored on that season alone.

Holdout discipline. The holdout has been evaluated once. Further tuning against it would turn it into a second test set, so the numbers in section 13 are final for this model, and the season page scores 2026-27 fixtures with exactly this bundle, unchanged.

13 Results

Headline metrics

HELD-OUT PERFORMANCE

Metric	Test (2023-24 + 2024-25, n = 699)	Holdout (2025-26,
model accuracy	56.1%	
always-home baseline	43.9%	
market accuracy (argmax of the price)	55.9%	
model log loss	0.9459	
market log loss (de-vigged)	0.9492	
model ranked probability score	0.1913	
market ranked probability score	0.1930	

The model beats always-home by about 12 accuracy points on test and 6 on holdout, sits 0.003 log loss ahead of the market on test and 0.003 behind it on holdout, which is small enough that the honest reading is level with the closing line rather than ahead of it.

Per-class recall

THE DRAW PROBLEM, IN NUMBERS

Class	Test recall	Test precision	Test support	Holdout recall	Holdout precision	H
Home	80.8%	57.5%	307	77.6%	52.3%	
Draw	0.0%	undefined	163	0.0%	undefined	
Away	62.9%	53.7%	229	52.4%	41.7%	

Draw precision is undefined on both splits because the model makes zero draw predictions. Across test and holdout, 261 matches ended in a draw and none were

called.

Test, n = 699				Holdout, n = 350			
	pred H	pred D	pred A		pred H	pred D	pred A
actual H	248	0	59	actual H	114	0	33
actual D	98	0	65	actual D	54	0	44
actual A	85	0	144	actual A	50	0	55

Figure 6 Confusion matrices. Rows are the actual result, columns the prediction; the shaded diagonal is correct. The middle column is empty on both splits. Drawn matches split roughly 60/40 into predicted-home and predicted-away on test and about evenly on holdout.

Feature importance

Top twenty by gain, classifier only, for the committed bundle. Gain is measured over training data, not over any evaluation split.

CLASSIFIER GAIN IMPORTANCE

#	Feature	Meaning	Gain
1	mkt_pow_A	market P(away), bias-adj	7.3
2	shots_roll15_diff	shots form	5.9
3	xgd_ewm_diff	xG-diff form (EWMA)	5.7
4	clv_D	draw line move	5.4
5	shots_std_diff	season shots/game	5.2
6	sot_std_diff	season SoT/game	5.1
7	xg_roll15_diff	xG form	5.1
8	close_p_H	closing P(home)	5.0
9	mkt_overround	bookmaker margin	4.8
10	clv_H	home line move	4.7
11	xg_ewm_diff	xG form (EWMA)	4.7
12	mkt_supremacy	market supremacy	4.6
13	corners_std_diff	season corners/game	4.6
14	mkt_p_D	market P(draw)	4.6
15	mkt_p_H	market P(home)	4.6
16	games_played_diff	games played gap	4.5
17	elo_diff	Elo rating gap	4.4
18	shots_ewm_diff	shots form (EWMA)	4.4
19	mkt_xg_away	market away xG	4.4
20	sot_ewm_diff	SoT form (EWMA)	4.3

Nine of the top twenty are market-derived. The model's own contribution concentrates in shot-volume and xG-form differentials rather than points or Elo; Elo difference ranks seventeenth despite being the strongest single correlate before training, because the market columns already carry most of what it says.

Does the squad data help?

ABLATION LADDER ON THE TEST SEASONS, CLASSIFIER ONLY, TUNED ON VALIDATION

Feature set	Columns	Test log loss	Test accuracy
form + Elo only	42	0.9494	56.1%
+ expected goals	50	0.9491	55.9%
+ market	69	0.9455	56.2%
+ squad (= all)	78	0.9462	56.1%

Squad data does not earn its place on this evidence. It makes the classifier marginally worse on the full test window and marginally better on 2024-25 alone (0.9813 to 0.9804), both deltas smaller than the gap between adjacent ladder rows. The likely explanation is that squad quality is already priced into the market features, which is where the ladder's real gain comes from. The features are retained because they are leak-free and cheap, not because they demonstrably help.

Rolling-origin check

THE FULL BLEND REFITTED PER SEASON ON EVERYTHING PRECEDING ITS TUNING BLOCK

Season	n	Model accuracy	Always home	Model log loss	Market log loss	Edge
2018-19	350	58.0%	47.7%	0.895	0.897	+0.00
2019-20	350	53.1%	46.3%	0.969	0.970	+0.00
2020-21	348	50.3%	38.2%	1.000	1.000	0.00
2021-22	350	58.9%	42.6%	0.945	0.941	-0.00
2022-23	350	56.3%	48.6%	0.953	0.956	+0.00
2023-24	349	59.0%	46.7%	0.911	0.917	+0.00
2024-25	350	53.7%	41.1%	0.978	0.981	+0.00



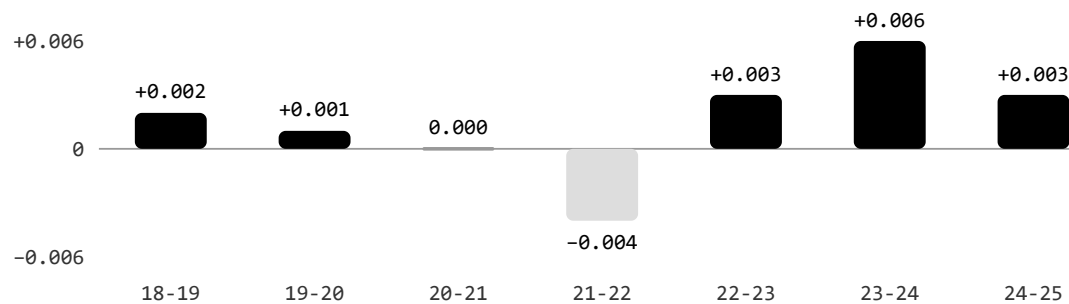


Figure 7 Edge over the market by season. Market log loss minus model log loss, refitting before each season. Positive in 5 of 7 seasons with one tie, mean +0.002: consistent with the test and holdout numbers, and consistent with no real edge. The 2020-21 dip is the empty-stadium season, which damaged the always-home baseline far more than the model.

14 Limitations and next steps

Limitations

- **It does not beat a calibrated bookmaker.** Holdout log loss is 1.019 against the market's 1.016, and the rolling-origin edge averages +0.002 across seven seasons, within noise for samples of about 350 matches.
- **It never predicts a draw.** 261 drawn matches across test and holdout, none called. Any use case requiring draw identification needs a different decision rule at minimum, probably a different formulation.
- **Scorelines come only from the goal model.** The most likely scoreline typically carries under 15% probability.
- **It is blind to mid-season change.** Injuries, suspensions, transfers, manager changes and European fixture load are not inputs; squad prices are frozen at gameweek 1.
- **Calibration is unstable mid-range** on 70-match buckets (Figure 5).
- **Cold-start matches are not scored in evaluation.** Rows where either team has fewer than three prior matches that season are dropped: 826 of 9,900. The season page does score such fixtures, and flags them.
- **Squad-value features are unproven** (section 13).

Next steps

- **Retrain on everything, separately.** The holdout is spent. A single final fit on all matches with meta-parameters fixed at their current values would be the correct

production model. It has deliberately not been done here, so that every number on this site belongs to one model with an out-of-sample record.

- **Add the signals the model is blind to.** An availability feed, a manager-change flag, days since a European fixture: the one category of information the bookmaker has and the model does not.
- **Reformulate the draw.** An ordinal model, or a scoreline-first model with the classifier demoted to a correction term. The committed blend already leans 70% on the goal model; making it the backbone rather than a blend partner is the natural direction.
- **Calibrate on a rolling window.** Temperature is fitted on a fixed two-season block; a trailing window would track drift in the home-advantage regime, which the Elo module handles and the calibrator does not.

15 Provenance

Headline metrics, feature importance, calibration deciles and the rolling-origin table are read from `data/ui_meta.json`, exported by the model notebook. Confusion matrices, per-class recall and precision, the reconstructed blend weights, the home-advantage curve and the split counts are derived from `data/ui_predictions.json` and the raw season files by `tools/build_site_stats.py` into `docs/data/site_stats.json`. Hyper-parameters, blend weight, ρ , temperature, the draw-seeking factor and the split definitions come from `models/config.json`. The ablation ladder is `data/ablation_result.json`. Outcome shares and feature correlations are `data/viz_stats.json`. The Man United sequence in section 5 is the 2015-16 season file. Every figure on this page carries a machine-readable pointer to its source, and a script in the repository checks each one against the artefact on every deploy.

Data: football-data.co.uk [3] and FPL Core Insights [4], used non-commercially. Code and pages under the MIT licence; the match data is not. Source: github.com/Daytoo77/pl-outcome-predictor [5].

Links

1. <https://www.football-data.co.uk/englandm.php>
2. <https://github.com/olbauday/FPL-Core-Insights>
3. <https://www.football-data.co.uk/englandm.php>
4. <https://github.com/olbauday/FPL-Core-Insights>
5. <https://github.com/Daytoo77/pl-outcome-predictor>